Step 1 — A Script That Sends One Message and Prints the Reply — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: before any chat, memory, or agents, this step proves the three foundations of the whole project work together: settings load from `.env`, logging is set up, and one real call to the model comes back. It's deliberately the smallest possible end-to-end slice — one message in, one reply out — so that if anything in your setup is wrong (a bad key, a missing package, a broken import), it fails *here*, in a handful of lines, where the cause is obvious. Each file also goes straight into its final `src/` folder from day one, so every later step only *adds* code rather than moving things around. **If not:** a wrong key or a broken import would first show up in Step 3, tangled together with ticket logic, and you'd be debugging three unrelated things at once.

After this step the project already runs: `python main.py` sends one message and prints the reply.

The Doc01 files

These three files are Doc01's Build Task code, **unchanged** — only the file locations and the `from src...` import line differ.

Story — `src/exceptions.py`: this file holds the project's own named error types. `load_config()` needs a way to say specifically "a required setting is missing" — a custom `MissingConfigError` — rather than raising a bare `Exception`. A named error lets the code that calls it catch *exactly* that problem and react (print a clean setup message and exit), while letting genuine bugs crash loudly instead of being swallowed. **If not:** `main.py` would have to catch every `Exception` to handle a missing key, which also silently hides real bugs like a typo in a variable name.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env. (Doc01's Build Task file.)
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

Story — `src/config.py`: this is the single place the whole project reads its settings from. On startup it loads `.env`, then checks that every required value (like `OPENAI_API_KEY`) is actually

present — and if one is missing, it stops immediately with a clear `MissingConfigError` instead of letting the program run half-configured. Centralizing this means the rest of the code can simply trust that a `Config` object is complete and valid. **If not:** every file would read `os.environ` itself, nobody would own the "is this set?" check, and a missing key would only surface later as a confusing `AuthenticationError` deep inside an API call.

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing.
# (Doc01's Build Task file — only the import line changed.)
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

`load_dotenv()` looks for `.env` starting from this file's folder and walking up, so it finds the `.env` in the repo root.

Story — `src/utils/logger.py`: this gives the whole project one logging setup, from a single `get_logger(name)` function: every file that calls it gets the same format (a timestamp, the file's name, the level) and the same output. The function also guards against adding a second handler to the same logger, which is the usual cause of every line printing twice. **If not:** each file would configure its own handler, the format would drift from file to file, and some lines would print twice.

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
```

```
# file name and the level on every line. (Doc01's Build Task file.)
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        logger.setLevel(level)

    return logger
```

Also create the empty `src/models/__init__.py` and `src/utils/__init__.py`, so both folders are importable packages.

Back to this step · Hint 1 · Hint 2 · Solution

Solution — typed, logged, and the key checked at startup

Story — `models/llm.py`: this is the only file in the project that talks to the OpenAI SDK directly — everything else goes through it, so if the API ever changes you have one place to fix. It defines the `MODEL` constant (so the model name isn't scattered as a string across the codebase), creates the client, and runs a startup key check that sends a tiny 1-token request: that costs almost nothing but exercises the exact endpoint every agent will later use, so a bad key is caught at launch. **If not:** a wrong key would first appear as a crash in the middle of the user's first real message, after they'd already typed it.

```
# src/models/llm.py
# The only file that talks to the OpenAI SDK: create_client(), the startup
# key check, and send_message().
import openai
from openai import OpenAI

MODEL = "gpt-4o-mini"

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client):
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True

def send_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply
```

Story — `main.py`: this is the entry point — the file you actually run — and it wires the foundations together in a deliberate order: load settings, set up the logger, check the key, then make the one real call. Each thing that can go wrong (missing config, a bad key) is caught and turned into a single clear line plus a clean `sys.exit(1)`, so the user sees a readable message, not a stack trace. That ordering is the template every later step follows. **If not:** a missing or wrong key would dump a long Python traceback that means nothing to a user, instead of a plain "check OPENAI_API_KEY in your .env file."

```
# main.py
# The file you run: python main.py
# Loads the settings, checks the key, sends one message, prints the reply.
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)
from src.utils.logger import get_logger

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    logger.info("Sending a test message to the model")
    history: list[dict] = []
    reply = send_message(client, history, "What's 2+2?")
    print(reply)

if __name__ == "__main__":
    main()
```

Run it (from the repo root): `python main.py`.

Expected output (your timestamp and the model's wording will differ):

```
2026-09-25 10:00:00,123 __main__ INFO Sending a test message to the model
2 + 2 equals 4.
```

The logger names itself `__main__` here, because the file you run is always named `__main__`.

Check the failure paths:

- Remove `OPENAI_API_KEY` from `.env` → `Setup problem: Required environment variable is missing: OPENAI_API_KEY`
- Set it to `sk-fake-key-123` → an `ERROR` log line, then `Login failed — check OPENAI_API_KEY in your .env file.`

Both exit with code 1 and no traceback.

The Step 1 tests

Story — `tests/unit/test_config.py`: this is the fast, no-cost safety net for your settings code: the same three checks as Doc01's `test_config.py` — a required setting that's missing, one that's present but empty, and a config that loads cleanly — all with no API call, so they run in milliseconds. It deliberately exercises `require_env()` with a made-up setting name (not your real keys), so running the test never moves or rewrites your actual `.env`. Each case prints `OK` or `WRONG`, so you never have to eyeball the output. **If not:** the first sign of a broken `.env` would be a confusing OpenAI error at runtime, instead of a clear `WRONG` from a test.

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env file is never
# touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "SUPPORTDESK_TEST_SETTING"


def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
```

```
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")


def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Story — `tests/integration/test_live_agents.py`: this holds the checks that genuinely need the real model — the ones a unit test can't cover. It starts with just two: one real reply comes back and contains the expected answer, and the startup key check correctly rejects a bad key. Every later step adds its own cases to this *same* file, so the integration suite grows with the project (the same style as Doc04's `test_chat_client.py`). **If not:** "it works" would only ever mean "it worked the one time I tried it by hand," with nothing to re-run after a change.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from openai import OpenAI

from src.config import load_config
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
```

```
    send_message,
)


def test_first_reply(client):
    reply = send_message(client, [], "What's 2+2?")
    result = "OK"
    if "4" not in reply:
        result = "WRONG"
    print(f"case 1 (first reply) -> {result}: {reply}")


def test_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    result = "OK"
    if check_api_key_at_startup(bad_client):
        result = "WRONG"
    print(f"case 2 (bad key)     -> {result}")


if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    test_first_reply(client)
    test_bad_key()
```

Also create the empty folders `tests/unit/` and `tests/integration/`. Tests run as modules, so no `__init__.py` is needed there.

Every test prints `OK` or `WRONG` — you never have to judge the output yourself. `WRONG` means: read that test function and the code it calls.

Run the tests (from the repo root):

```
python -m tests.unit.test_config
python -m tests.integration.test_live_agents
```

Expected output of `test_config`:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

Expected output of `test_live_agents` (the model's wording varies):

```
case 1 (first reply) -> OK: 2 + 2 equals 4.
case 2 (bad key)     -> OK
```